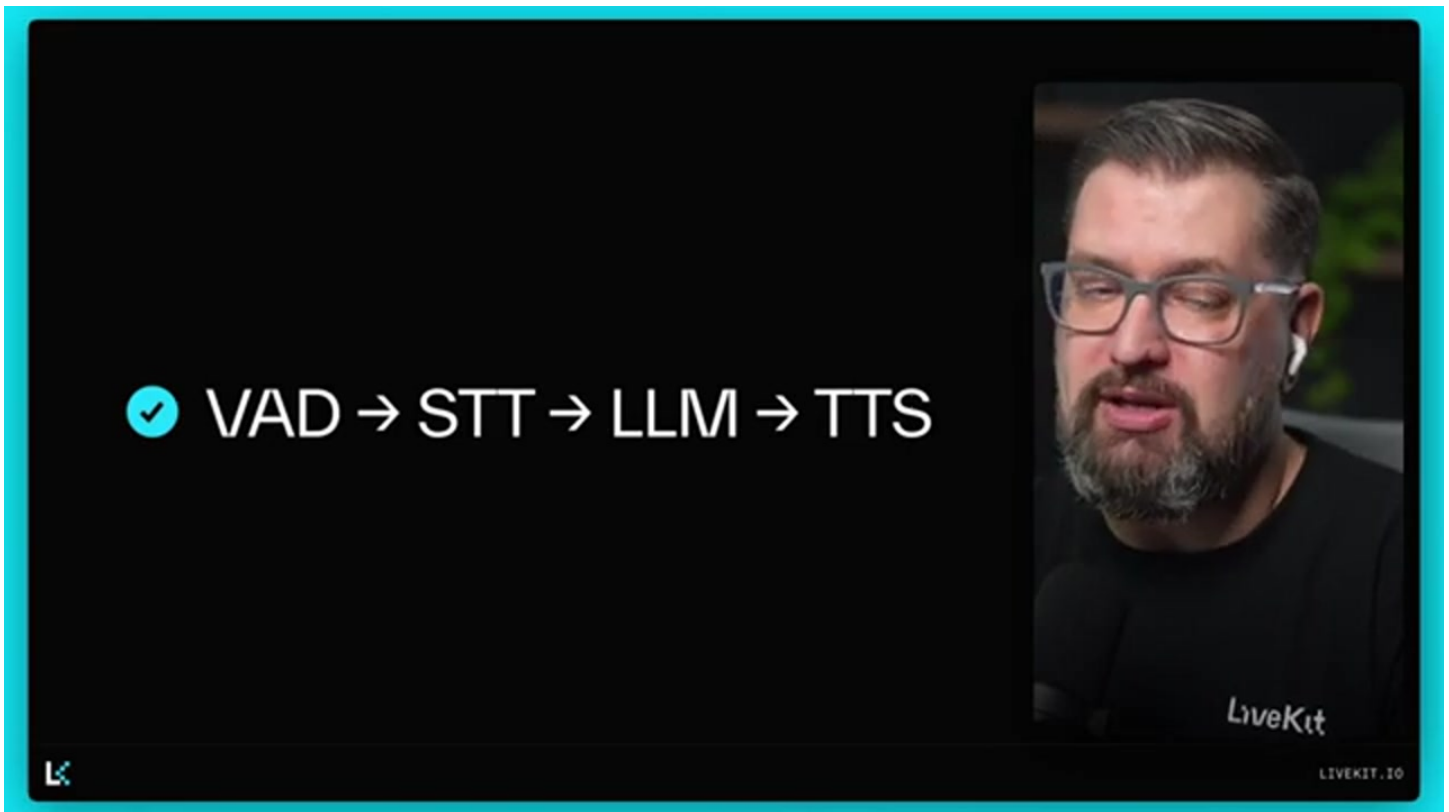


Lesson 2: Semantic Turn Detection

Section: LiveKit Voice AI Course

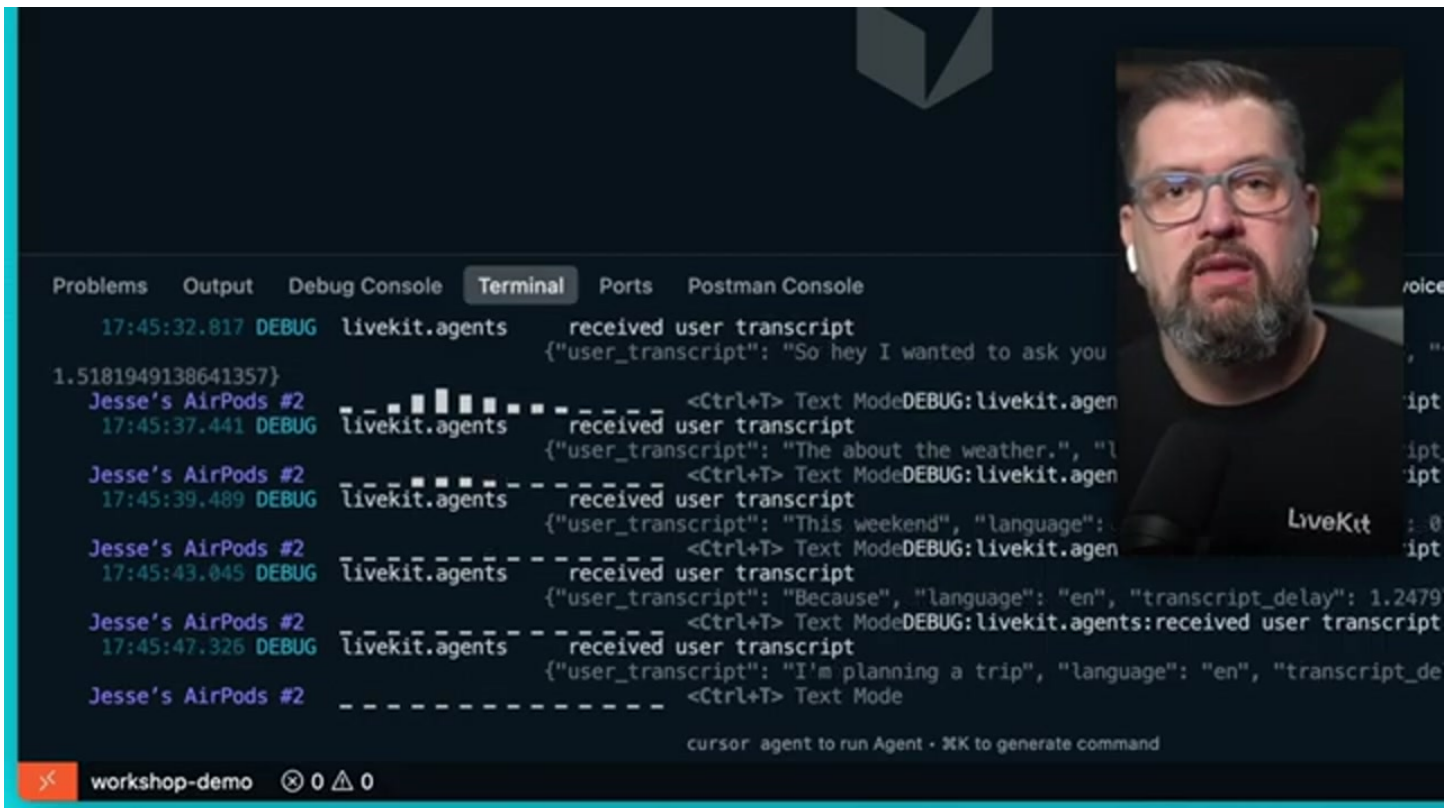
Summary: Coverage of Lesson 2: Semantic Turn Detection (13:24 - 18:46)

■ [13:24 - 13:44]



early. Hey, I wanted to ask you um about >> Sure. What would you like? >> No, about the weather uh this weekend. Um because I'm um >> Oh, no. I'm planning a trip

■ [13:44 - 14:04]



The image shows a video call interface with a man on the right and a terminal window on the left. The man has a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. The terminal window displays a series of debug logs from 'livekit.agents' showing the receipt of user transcripts. The transcripts are: 'So hey I wanted to ask you', 'The about the weather.', 'This weekend', 'Because', and 'I'm planning a trip'. Each transcript is followed by a '<Ctrl+T> Text Mode' command. The terminal also shows a 'cursor: agent to run Agent - %K to generate command' message at the bottom.

```
17:45:32.817 DEBUG livekit.agents received user transcript
{"user_transcript": "So hey I wanted to ask you
1.5181949138641357}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agen
17:45:37.441 DEBUG livekit.agents received user transcript
{"user_transcript": "The about the weather.", "l
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agen
17:45:39.489 DEBUG livekit.agents received user transcript
{"user_transcript": "This weekend", "language":
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agen
17:45:43.045 DEBUG livekit.agents received user transcript
{"user_transcript": "Because", "language": "en", "transcript_delay": 1.2479
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:received user transcript
17:45:47.326 DEBUG livekit.agents received user transcript
{"user_transcript": "I'm planning a trip", "language": "en", "transcript_de
Jesse's AirPods #2 <Ctrl+T> Text Mode

cursor: agent to run Agent - %K to generate command
```

workshop-demo 0 0

>> Oh, no. I'm planning a trip to um Sounds exciting. >> So that's what happens when your agent relies only on voice activity detection. It hears a pause and it thinks that you're done. But humans pause mid-sentence all the time. We think, we breathe, we change direction. Uh, a

■ [14:04 - 14:24]

Semantic turn detection



LiveKit

LIVEKIT.IO

we breathe, we change direction. Uh, a good conversational agent needs to understand when someone is actually finished speaking. And so turn detection decides when the agent should speak and when it should stay quiet. And without it, you would get interruptions. And the agent starts answering at every single pause, even if the human is still

■ [14:24 - 14:44]

Semantic turn detection



LiveKit

LIVEKIT.IO

pause, even if the human is still mid-sentence. And so in this lesson, you'll add a semantic turn detector that dramatically improves conversational quality. Now, voice activity detection spots human patterned audio, but humans pause, restart sentences, and trail off all the time. And so, VAD sees a pause and it

VAD vs Semantic detection



LiveKit

LIVEKIT.IO

time. And so, VAD sees a pause and it thinks that you're done talking. But semantic detection looks at all of the the meaning of the sentence and it asks, does this feel like a complete thought? The most important job of a turn detector is to reduce unwanted interruptions. It also improves accuracy for the ST engine since it gets complete

Turn detection

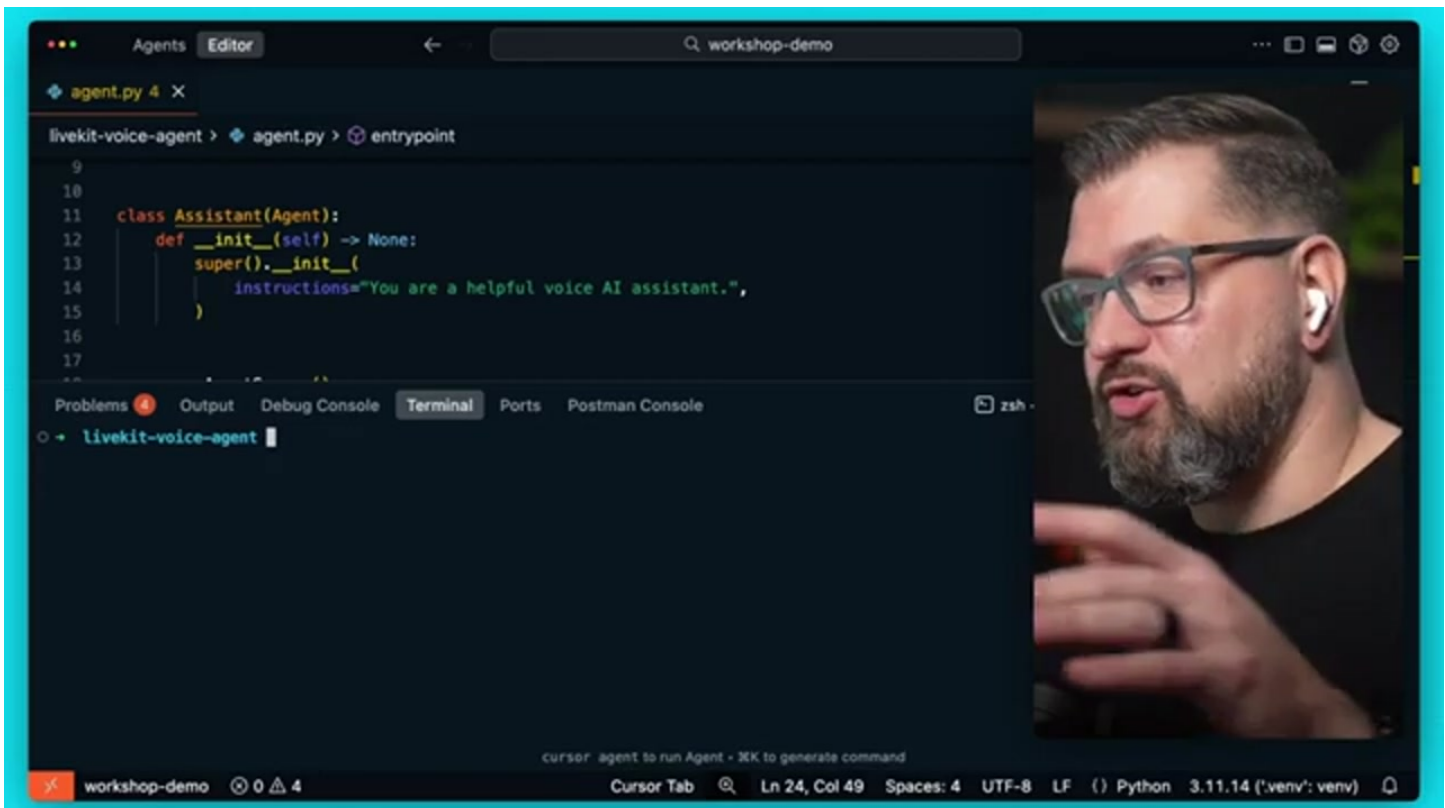
- Reduces unwanted interruptions
- Improves STT accuracy



LIVEKIT.IO

for the ST engine since it gets complete utterances instead of fragments and the latency impact is minimal only around 20 milliseconds. So the first thing that we'll need to do is add the turn detector extra to our environment. So in Python an extra is an optional uh set of dependencies that a package can include.

■ [15:24 - 15:44]



dependencies that a package can include. So we're going to UV add lifkit agents turn detector. So, this pulls in a multilingual model uh that you'll reference in the code, right? So, we'll go ahead and import this multilingual model from our uh turn detector plugin. We'll go down to our agent session here and then we'll add into our agent

■ [15:44 - 16:04]

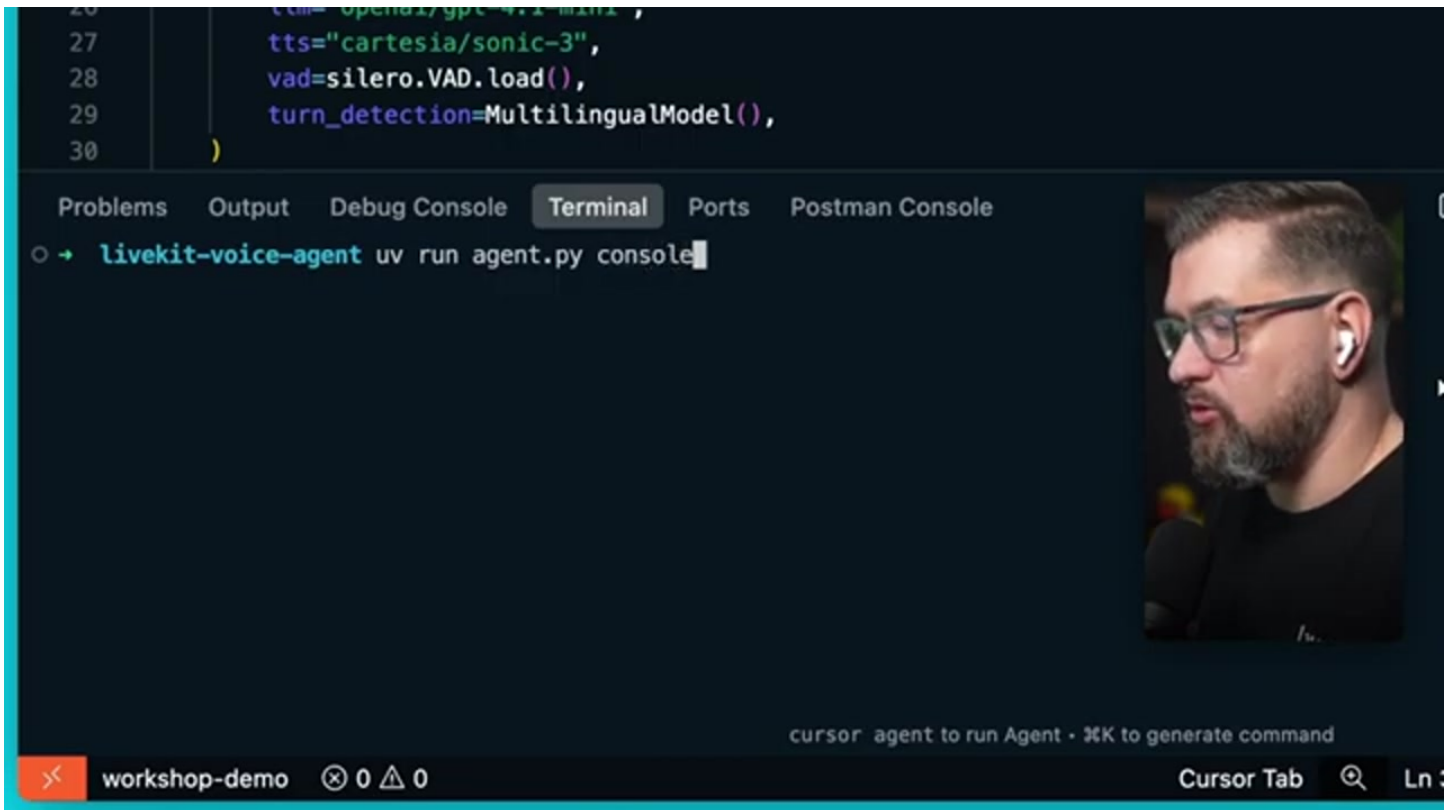
```
agent.py ●
livekit-voice-agent > agent.py

22 @server rtc_session()
23 async def entrypoint(ctx: JobContext):
24     session = AgentSession(
25         stt="assemblyai/universal-streaming:en",
26         llm="openai/gpt-4.1-mini",
27         tts="cartesia/sonic-3",
28         vad=silero.VAD.load(),
29         turn_detection=MultilingualModel(), turn_detection=
30     )
31
32     await session.start(
33         agent=Assistant(),
34         room=ctx.room,
35         room_options=room_io.RoomOptions(
36             audio_input=room_io.AudioInputOptions(
37                 noise_cancellation=noise_cancellation.BVC(),
38             ),
39         ),
40     )
```



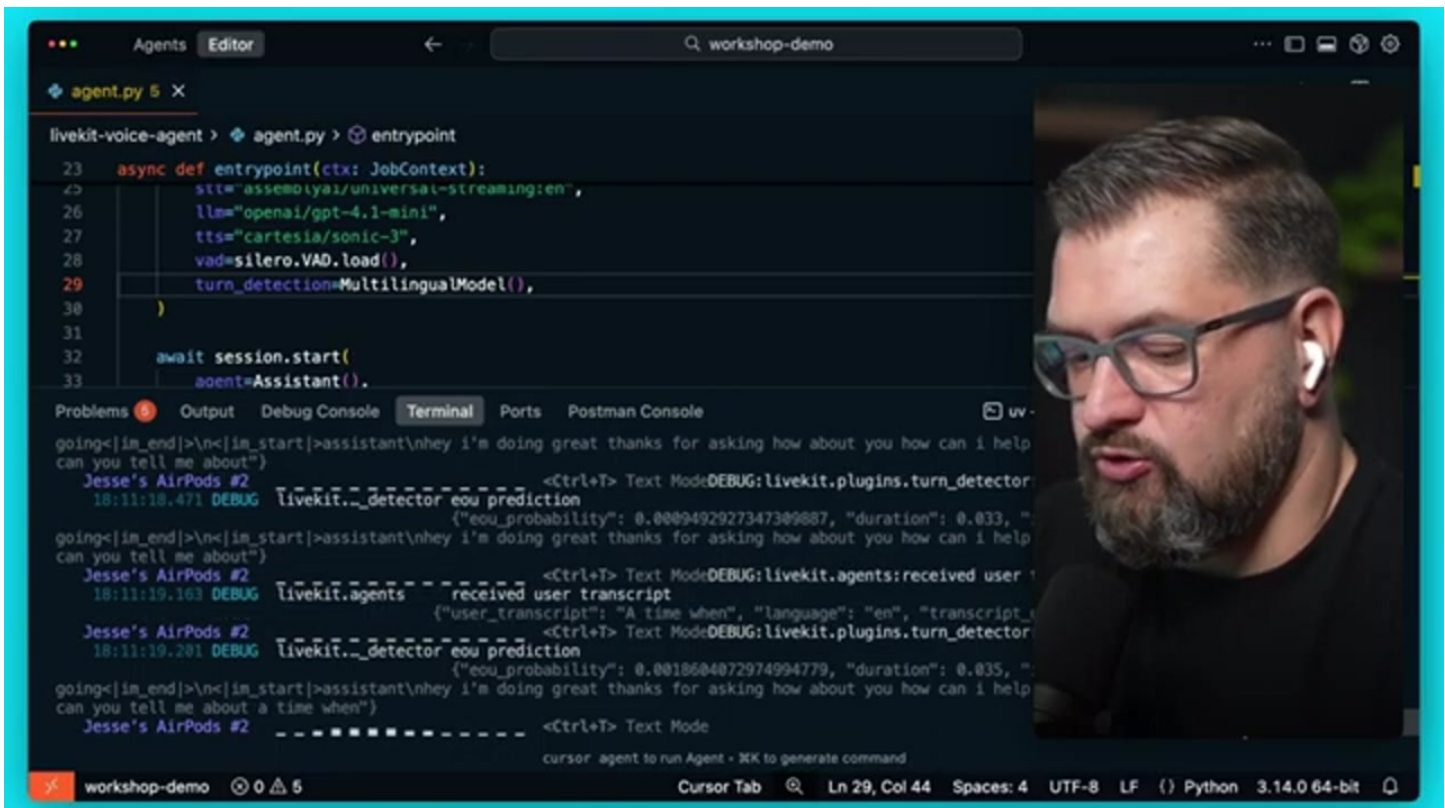
and then we'll add into our agent session this uh turn detection and pass it the multilingual model. Now, save that. And that's all that you have to do. The session now emits turn events when the semantic model decides a speaker is finished. So, let's go ahead and test it out. All right, let's open up our terminal. We'll run `uv run agent.py` and console to run it in the

■ [16:04 - 16:24]



agent.py and console to run it in the console. Let's see if it's any better. Yeah. Hey, how's it going? >> Hey, I'm doing great. Thanks for asking. >> How about Can you tell me about um about a time when uh you you struggled to come up with

■ [16:24 - 16:44]



uh you you struggled to come up with something to say? >> Sure. Even though I'm designed to generate responses quickly, there are times when a question is very vague or complex and it can be challenging to provide me. >> Let's try to interrupt him and like just like tell me something else.

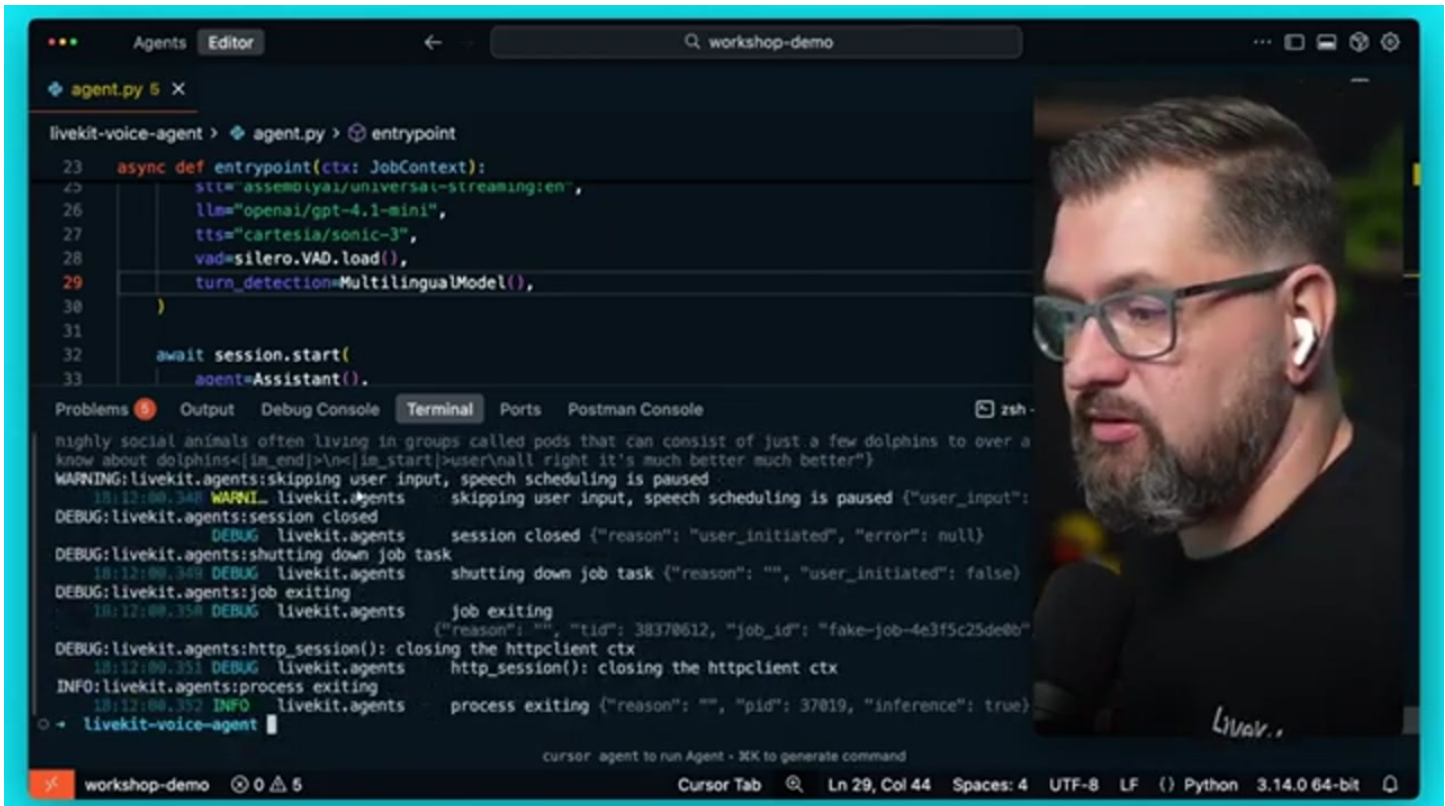
■ [16:44 - 17:04]

The screenshot shows a VS Code window with the following components:

- Editor:** A Python file named `agent.py` is open. The code defines an `entrypoint` function that initializes a `livekit-agent` with various settings like `stt`, `llm`, `tts`, `vad`, and `turn_detection`.
- Terminal:** Displays a conversation log. It shows a user transcript, a system message about generating responses, and a user transcript asking for something else to say.
- Status Bar:** Shows the current cursor position as `Ln 29, Col 44` and the file encoding as `UTF-8`.

like tell me something else. Got it. Switching gears. Did you know that octopuses have three hearts in blue blood? Two of their hearts. >> Tell me um something about um dolphins. >> Absolutely. Dolphins are incredibly intelligent marine. >> Much better. Much better. Let's stop it.

■ [17:04 - 17:24]



The screenshot shows a video call interface. On the right, a man with a beard and glasses is speaking into a microphone. On the left, a code editor window titled 'workshop-demo' is open, showing a Python file named 'agent.py'. The code defines an 'entrypoint' function for a 'livekit-voice-agent'. The function sets up an OpenAI GPT-4.1-mini model, a Cartesia TTS engine, and a Silero VAD for turn detection. The terminal output shows the agent running, processing a user input about dolphins, and then pausing speech scheduling. The status bar at the bottom indicates the file is 'Ln 29, Col 44' and the environment is 'Python 3.14.0 64-bit'.

```
livekit-voice-agent > agent.py > entrypoint
23 async def entrypoint(ctx: JobContext):
24     stt="asemoai/universal-streaming:en",
26     llm="openai/gpt-4.1-mini",
27     tts="cartesia/sonic-3",
28     vad=silero.VAD.load(),
29     turn_detection=MultilingualModel(),
30 )
31
32 await session.start(
33     agent=Assistant(),

```

highly social animals often living in groups called pods that can consist of just a few dolphins to over a
know about dolphins<|im_end>\n<|im_start>user\nall right it's much better much better")
WARNING:livekit.agents:skipping user input, speech scheduling is paused
18:12:00.348 WARNING livekit.agents skipping user input, speech scheduling is paused ("user_input":
DEBUG:livekit.agents:session closed
DEBUG livekit.agents session closed {"reason": "user_initiated", "error": null}
DEBUG:livekit.agents:shutting down job task
18:12:00.348 DEBUG livekit.agents shutting down job task {"reason": "", "user_initiated": false}
DEBUG:livekit.agents:job exiting
18:12:00.358 DEBUG livekit.agents job exiting
{"reason": "", "tid": 38370612, "job_id": "fake-job-4e3f5c25de0b"
DEBUG:livekit.agents:http_session(): closing the httpclient ctx
18:12:00.351 DEBUG livekit.agents http_session(): closing the httpclient ctx
INFO:livekit.agents:process exiting
18:12:00.352 INFO livekit.agents process exiting {"reason": "", "pid": 37019, "inference": true}
+ livekit-voice-agent

cursor: agent to run Agent - RK to generate command

workshop-demo 0 0 5 Cursor Tab Ln 29, Col 44 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

>> Much better. Much better. Let's stop it. So, this single change dramatically improves conversational quality. The agent feels more patient and natural. And this prepares you for more advanced latency optimization in later lessons. So let's talk about some best practices. Now a few things that you can keep in mind when using turn detection is make

■ [17:24 - 17:44]

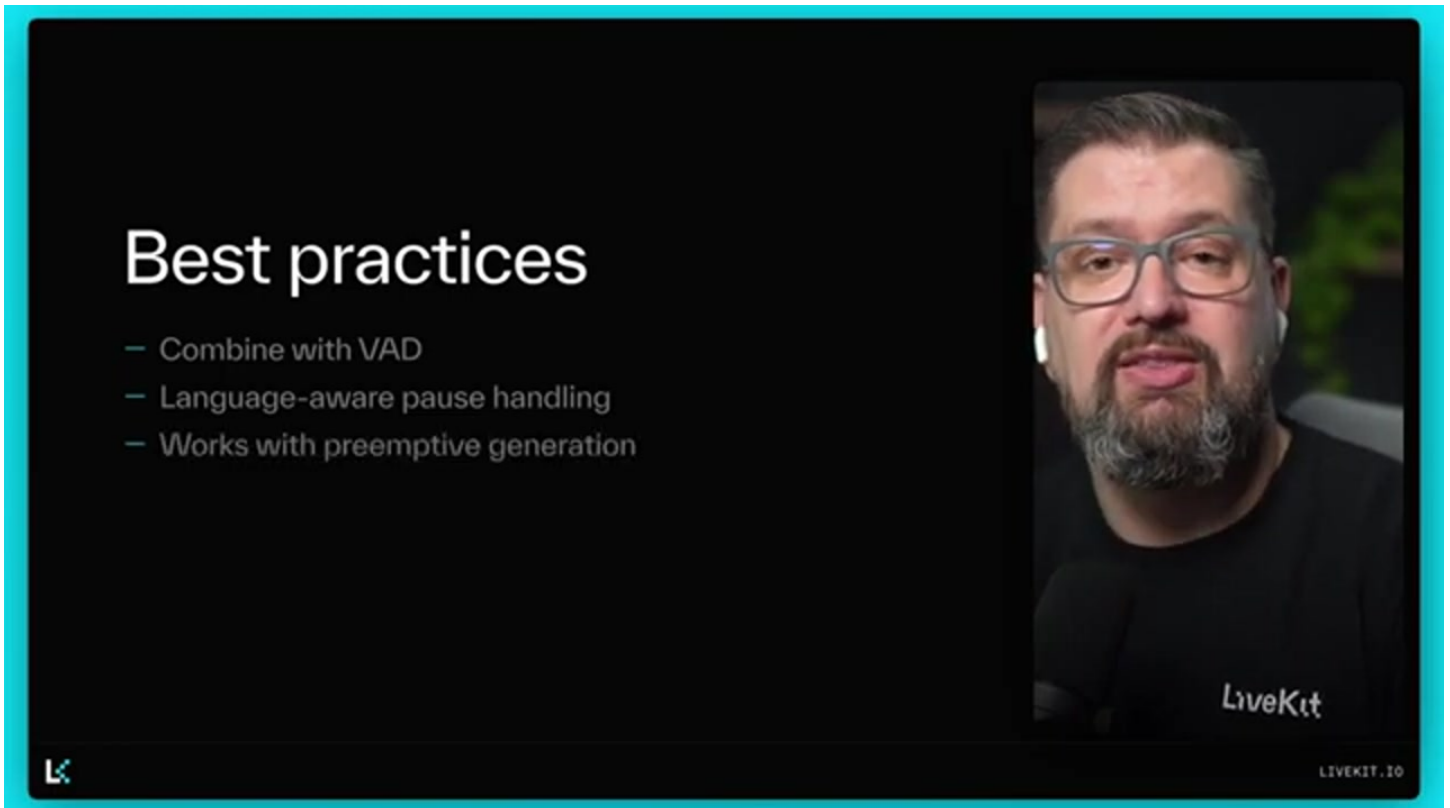
Best practices



LiveKit

LIVEKIT.IO

mind when using turn detection is make sure that you combine it with VAD and noise control for best results. Different languages have different pause patterns as well and semantic models uh help to normalize this. It also coordinates with preemptive generation, which we're going to talk about in a future lesson. And this allows the LLM



The video frame displays a presentation slide with a black background and white text. The slide title is "Best practices". Below the title are three bullet points: "– Combine with VAD", "– Language-aware pause handling", and "– Works with preemptive generation". In the bottom right corner of the slide, there is a small "LIVEKIT" logo. To the right of the slide, a man with a beard and glasses is speaking into a microphone. He is wearing a black t-shirt with the "LiveKit" logo. The video frame has a cyan border. In the bottom left corner of the frame, there is a small "L" logo, and in the bottom right corner, the text "LIVEKIT.IO" is visible.

Best practices

- Combine with VAD
- Language-aware pause handling
- Works with preemptive generation

LIVEKIT

LIVEKIT.IO

future lesson. And this allows the LLM to start planning while it's still waiting for a clear end of turn. Here are a few scenarios worth testing. First, try rapidfire short sentences and compare that to a single long sentence with natural pauses. See how the agent handles each of these. Uh, second, add

Ideas for testing

- Short sentences vs long pauses
- Background noise



handles each of these. Uh, second, add some background noise. turn on a TV or have someone talking nearby. Does the agent stay focused on your voice? And third, if you speak multiple languages, try switching languages midterm. The multilingual model should handle this gracefully. Add these to your testing backlog. The more scenarios you try, the more

Ideas for testing

- Short sentences vs long pauses
- Background noise
- Mid-sentence language switch



LIVEKIT.IO

more scenarios you try, the more confident you'll be in how your agent behaves. So, with semantic turn detection in place, your agent no longer jumps in at every pause. It waits for complete thoughts before responding. In the next lesson, you'll give an agent a distinctive personality, uh, change its voice, and add fallback providers so

■ [18:44 - 18:46]

Personality & fallbacks



LiveKit

LIVEKIT.IO

voice, and add fallback providers so that it survives outages. Now that